

对抗持续学习项目报告

Adversarial Continual Learning

<https://github.com/KindofCrazy/LibContinual-ACL/tree/yk>

221900275 杨可
221900079 徐伟
221900288 陈泓锦

2024年 机器学习



南京大學

2024年6月30日

对抗持续学习项目报告

Adversarial Continual Learning

<https://github.com/KindofCrazy/LibContinual-ACL/tree/yk>

2024年 机器学习

一、相关复现背景介绍

- 1.1 持续学习简介 (Continual learning)
- 1.2 对抗学习简介 (Adversarial learning)
- 1.3 潜在空间分解 (Latent Space Factorization)

二、对抗持续学习介绍

- 2.1 在 ACL 中避免遗忘
- 2.2 评估指标

三、复现的过程和难点

- 3.1 文件目录
- 3.2 Dataset and Dataloader
- 3.3 Training Process
- 3.4 Model

四、复现结果

- 4.1 实验环境
- 4.2 实验结果
- 4.2 日志

五、遇到的问题与解决

- 5.1 数据加载的设置
- 5.2 backbone 与 buffer 的设置
- 5.3 训练

六 思考

- 6.1 模型的设置
- 6.2 Shared 模块的作用

一、相关复现背景介绍

1.1 持续学习简介 (Continual learning)

现有的持续学习方法大致可分为三类:基于记忆的方法、基于结构的方法和基于正则化的方法。

- 基于记忆的方法:这类方法通过显式或隐式地存储以前的经验来减轻遗忘, 其中前者的原始样本被保存到记忆中进行排练, 而后者是GAN或自动编码器等生成模型合成它们进行伪排练。这些方法允许在i.i.d数据上同时进行多任务学习, 这可以显著减少遗忘。最近一项关于CL的微小情景记忆的研究比较了GEM、A-GEM、MER和ER-RES等方法。对于ER-RES, 他们使用单遍数据进行储层采样。与随机选择相比, 储层抽样对于长输入数据是一种更好的抽样策略。
- 基于结构的方法:这些方法利用模块化, 并尝试将推理定位到网络的一个子集, 如列、神经元、参数掩码。通过存储学习模块来保持先前任务的性能, 同时通过增加新的模块来适应新的任务。例如, 渐进式神经网络(Progressive Neural Nets, PNNs)在静态增长架构的同时保留与先前冻结模块的横向连接, 从而以参数数量的二次尺度为代价保证零遗忘。提出了动态扩展网络(DEN), 其中网络容量根据任务相关性增长, 通过拆分/复制最重要的神经元, 同时对它们进行时间戳, 使它们始终保持可访问和可重新训练。尽管这种策略引入了计算成本, 但在需要学习大量任务且不能假设固定容量的持续学习场景中, 这种策略是不可避免的。
- 正则化方法:在这些方法中, 假设学习能力是固定的, 并进行持续学习, 这样, 如果参数的变化导致先前任务的性能下降, 则可以控制和减少或防止参数的变化。因此, 对于参数选择, 必须定义一个权重重要度量概念来对参数进行优先级排序使用。例如, 受贝叶斯学习的启发, 在弹性权重巩固(elastic weight consolidation, EWC)方法中, 重要的参数是在Fisher信息矩阵中具有最高的参数。HAT在重要参数上学习注意掩模。尽管这些方法在最大限度地利用固定容量方面取得了成功, 但它们往往受到任务数量的限制。

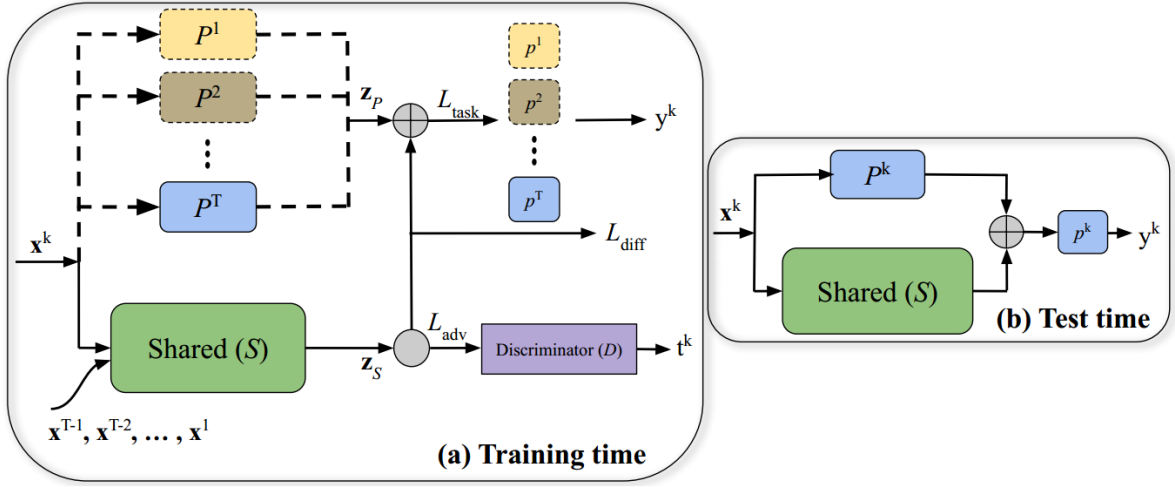
1.2 对抗学习简介 (Adversarial learning)

对抗性学习已被用于不同的问题, 如生成模型、对象组成、表示学习、领域适应、主动学习等。对抗网络的使用使模型能够通过调整来解决极大极小优化问题, 从而以完全可微的方式进行训练。潜在空间的对抗性学习在领域适应、主动学习和表征学习中得到了广泛的研究。

1.3 潜在空间分解 (Latent Space Factorization)

在机器学习文献中, 多视图学习旨在构建和/或使用不同的视图或模式以获得更好的学习性能。解决多视图学习的方法要么旨在最大化不同数据视图的相互一致性, 要么专注于通过使用典型相关分析和聚类、高斯过程等, 假设输入视图是从该潜在子空间生成的, 从而获得由多个视图共享的潜在子空间。因此, 对于不同的数据模式, 将潜在空间分解为共享部分和私有部分的概念得到了广泛的探讨。

二、对抗持续学习介绍



ACL 考虑学习一系列 T 个数据分布的问题，这些数据分布表示为 $D^{tr} = \{D_1^{tr}, \dots, D_T^{tr}\}$ ，其中 $D_k^{tr} = \{(\mathbf{X}_i^k, \mathbf{Y}_i^k, T_i^k)_{i=1}^{n_k}\}$ 是任务 k 的数据分布，具有 n 个样本对输入 ($\mathbf{X}^k \in \mathcal{X}$)、输出标签 ($\mathbf{Y}^k \in \mathcal{Y}$) 和任务标签 ($T^k \in \mathcal{T}$)。目标是依次学习模型 $f_\theta: \mathcal{X} \rightarrow \mathcal{Y}$ ，使其能够将每个任务输入映射到其目标输出，同时保持在所有先前任务上的性能。ACL 旨在通过学习一个由任务特定的潜在空间和所有任务的任务无关特征空间组成的分离潜在空间表示来实现这一点，以增强更好的知识转移，并更好地避免先前知识的灾难性遗忘。

ACL 在每个空间中以不同的方式缓解灾难性遗忘。对于任务无关的特征空间，ACL 假设有一个有限的记忆预算 $\mathcal{M}^k$ ，它存储了来自任务 k 之前的每个任务的 m 个样本 $x_{i=1..m} \sim D_{j=1..k-1}^{tr}$ 。

ACL 开始通过将 f_θ^k 学习为从 $\mathbf{X}^k$ 到 $\mathbf{Y}^k$ 的映射。对于 C 类分类任务，对于交叉熵损失，这对应于：

$$\mathcal{L}_{\text{task}}(f_\theta^k, \mathbf{X}^k, \mathbf{Y}^k, \mathcal{M}^k) = -\mathbb{E}_{(x^k, y^k) \sim (\mathbf{X}^k, \mathbf{Y}^k) \cup \mathcal{M}^k} \sum_{c=1}^C \mathbb{1}_{[c=y^k]} \log(\sigma(f_\theta^k(x^k))) \quad (1)$$

其中 σ 是 softmax 函数，为了简洁起见，下标 $i = \{1, \dots, n_t\}$ 被省略。在学习一系列任务的过程中，一个理想的 f^k 是一个模型，它将输入映射到两个独立的潜在空间，其中一个包含所有任务的共享特征，另一个则保留为每个任务的私有特征。特别地，ACL 希望将潜在空间解缠到所有任务共享的信息 ($\mathbf{z}_S$) 和每个任务的独立或私有信息 ($\mathbf{z}_P$)，并使它们的拼接尽可能地不同，然后通过任务特定的头输出所需的目标。

为此，ACL 引入了一个称为 Shared ($S_{\theta_S}: \mathcal{X} \rightarrow \mathbf{z}_S$) 的映射，并训练它生成欺骗对抗判别器 D 的特征。相反，对抗判别器 ($D_{\theta_D}: \mathbf{z}_S \rightarrow \mathcal{T}$) 试图通过任务标签 ($T^k \in \{0, \dots, T\}$) 来分类生成的特征。当判别器被训练成最大化分配正确任务标签的概率时，实现这一点，同时 S 被训练成通过最小化 $\log(D(S(x^k)))$ 来迷惑判别器。这对应于以下极小极大博弈的 T 类分类交叉熵对抗损失：

$$\mathcal{L}_{\text{adv}}(D, S, \mathbf{X}^k, T^k, \mathcal{M}^k) = \min_S \max_D \sum_{k=0}^T \mathbb{1}_{[k=t^k]} \log(D(S(x^k))) \quad (2)$$

请注意，额外的标签零与随机生成的噪声特征 $\mathbf{z}'_S \sim \mathcal{N}(\mu, \Sigma)$ 相关联。特别地，ACL 在不同的模式下使用对抗学习，这种模式出现在大多数与生成对抗网络相关的工作中，因此输入数据分布的生成建模在这里没有被使用，因为最终任务是学习一个判别表示。

为了方便训练 S ，ACL 使用梯度反转层，它优化映射以直接最大化判别器损失 ($\mathcal{L}_{\text{tasks}} = -\mathcal{L}_D$)。实际上，它在前向传播期间充当恒等函数，但在反向传播期间否定其输入并反转梯度。当 S 能够生成 D 无法再预测正确任务标签的特征，使 $\mathbf{z}_S$ 尽可能成为任务无关时， S 和 D 的训练就完成了。私有模块 ($P_{\theta_P} : \mathcal{X} \rightarrow \mathbf{z}_P$) 试图通过仅学习与当前任务特定的特征来适应任务无关的特征，而这些特征不存在于 $\mathbf{z}_S$ 中。ACL 进一步通过使用正交约束将 $\mathbf{z}_S$ 和 $\mathbf{z}_P$ 因子化，对应于：

$$\mathcal{L}_{\text{diff}}(S, P, \mathbf{X}^k, \mathcal{M}^k) = \sum_{k=1}^T \|(S(x^k))^{\top} P^k(x^k)\|_F^2, \quad (3)$$

其中 $\|\cdot\|_F$ 是 Frobenius 范数，它是所有编码当前任务和记忆样本的 P 模块编码特征的总和。然后，使用任务特定的多层感知头预测每个任务的最终输出预测，该头以 $\mathbf{z}_P$ 与 $\mathbf{z}_S$ ($\mathbf{z}_P \oplus \mathbf{z}_S$) 拼接作为输入。综合起来，这些损失构成 ACL 的完整目标，如下所示：

$$\mathcal{L}_{\text{ACL}} = \lambda_1 \mathcal{L}_{\text{adv}} + \lambda_2 \mathcal{L}_{\text{task}} + \lambda_3 \mathcal{L}_{\text{diff}}, \quad (4)$$

其中 λ_1 、 λ_2 和 λ_3 是控制每个组成部分效果的正则化参数。ACL 的完整算法在算法1中给出。

2.1 在 ACL 中避免遗忘

灾难性遗忘发生在通过一系列任务学习的表示发生变化，以便学习当前任务，从而导致先前任务的性能下降。ACL 方法的主要见解是为一系列任务学习的常规单一表示分解为两部分：一部分是必须不变的，因为它包含任务特定的特征，如果没有这些特征，完全的性能恢复是不可能的；另一部分是更不容易变化的，因为它包含所有任务的核心结构。

为了在第一部分（私有特征）中完全防止灾难性遗忘，ACL 使用可以存储到内存中的紧凑模块。如果因子化成功执行，第二部分将高度免于遗忘。然而，ACL 实验证明，当任务之间几乎没有重叠或存在较大领域转变时，解缠不能完全实现，使用包含少量旧数据样本的小重放缓冲区有助于保持较高的 ACC 值并减轻遗忘。

2.2 评估指标

在每次训练新任务后，ACL 评估所有先前任务上的结果模型。为了衡量 ACL 性能，ACL 使用 ACC 作为所有任务的平均测试分类准确率。为了衡量遗忘，ACL 报告反向传递 (BWT)，它指示学习新任务对先前任务性能的影响。当 $\text{BWT} < 0$ 时直接报告灾难性遗忘， $\text{BWT} > 0$ 表明学习新任务有助于之前的任务。

$$\text{BWT} = \frac{1}{T-1} \sum_{i=1}^{T-1} (R_{T,i} - R_{i,i}), \quad \text{ACC} = \frac{1}{T} \sum_{i=1}^T R_{T,i} \quad (5)$$

Algorithm 1 Adversarial Continual Learning (ACL)

```
1: function TRAIN( $\theta_P, \theta_S, \theta_D, \mathcal{D}^{tr}, \mathcal{D}^{ts}, m$ )
2:   Hyper-parameters:  $\lambda_1, \lambda_2, \lambda_3, \alpha_S, \alpha_P, \alpha_D$ 
3:    $R \leftarrow 0 \in \mathbb{R}^{T \times T}$ 
4:    $\mathcal{M} \leftarrow \{\}$ 
5:    $f_\theta^k = f(\theta_S \oplus \theta_P)$ 
6:   for  $k = 1$  to  $T$  do
7:     for  $e = 1$  to epochs do
8:       Compute  $\mathcal{L}_{adv}$  for  $S$  using  $(x, t) \in \mathcal{D}_k^{tr} \cup \mathcal{M}$ 
9:       Compute  $\mathcal{L}_{task}$  using  $(x, y) \in \mathcal{D}_k^{tr} \cup \mathcal{M}$ 
10:      Compute  $\mathcal{L}_{diff}$  using  $P^k, S$ , and  $x \in \mathcal{D}_k^{tr}$ 
11:       $\mathcal{L}_{ACL} = \lambda_1 \mathcal{L}_{adv} + \lambda_2 \mathcal{L}_{task} + \lambda_3 \mathcal{L}_{diff}$ 
12:       $\theta'_S \leftarrow \theta_S - \alpha_S \nabla \mathcal{L}_{ACL}$ 
13:       $\theta'_{P_t} \leftarrow \theta_{P^k} - \alpha_{P^k} \nabla \mathcal{L}_{ACL}$ 
14:      Compute  $\mathcal{L}_{adv}$  for  $D$  using  $(S(x), t)$  and  $(\mathbf{z}' \sim \mathcal{N}(\mu, \Sigma), t = 0)$ 
15:       $\theta'_D \leftarrow \theta_D - \alpha_D \nabla \mathcal{L}_{adv}$ 
16:    end for
17:     $\mathcal{M} \leftarrow \text{UPDATEMEMORY}(\mathcal{D}_k^{tr}, \mathcal{M}, C, m)$ 
18:    Store  $\theta_{P^k}$ 
19:     $f_\theta^k \leftarrow f(\theta'_S \oplus \theta'_{P_t})$ 
20:     $R_{k, \{1 \dots k\}} \leftarrow \text{EVAL}(f_\theta^k, \mathcal{D}_{\{1 \dots k\}}^{ts})$ 
21:  end for
22: end function
```

```
function UPDATEMEMORY( $\mathcal{D}_k^{tr}, \mathcal{M}, C, m$ )
   $s \leftarrow \frac{m}{C} \triangleright s := \#$  of samples per class
  for  $c = 1$  to  $C$  do
    for  $i = 1$  to  $n$  do
       $(x_i^k, y_i^k, t_i^k) \sim \mathcal{D}_k^{tr}$ 
       $\mathcal{M} \leftarrow \mathcal{M} \cup (x_i^k, y_i^k, t_i^k)$ 
    end for
  end for
  return  $\mathcal{M}$ 
end function
function EVAL( $f_\theta^k, \mathcal{D}_{\{1 \dots k\}}^{ts}$ )
  for  $i = 1$  to  $k$  do
     $R_{k,i} = \text{Accuracy}(f_\theta^k(x, t), y) \text{ for } (x, y, t) \in \mathcal{D}_i^{ts}$ 
  end for
  return  $R$ 
end function
```

其中 $R_{n,i}$ 是在顺序完成第 n 个任务后在任务 i 上的测试分类准确率。ACL 还基于在网络架构增长或重放缓冲区中使用的内存来比较方法。

三、复现的过程和难点

3.1 文件目录

Dataset and Dataloader:

- /core/data/dataloader.py
- /core/data/dataset.py

Training Process:

- /core/model/trainer.py

Model:

- /core/model/ac1.py

Config File:

- /config/ac1.yaml

README.md:

- /reproduce/ac1/README.md

Checkpoints:

- /checkpoints

3.2 Dataset and Dataloader

需要对数据集和数据加载器进行修改，以适应 ACL 算法的需求。重写 `dataloader.py` 中的映射方式，使每个任务均为 5 个类别，且每个任务中的 `label` 均为 `[0,1,2,3,4]`。接下来，重写 `dataset.py` 中的数据集类，特别是 `ContinualDataset` 类的 `create_loader` 函数，将数据集分为多个任务，每个任务包含 5 个类别。同时在生成数据集时，加入 `tt` 与 `td` 参数。

3.3 Training Process

需要对训练过程进行大幅度修改，以适应 ACL 算法的需求。首先要重写 `trainer.py` 中的训练过程。在 `__init__` 方法中，`optimizer` 和 `scheduler` 需要自己实现。在 `train_loop` 方法中，需要自己实现训练过程，并且在每个 `epoch` 后手动更新学习率。在 `_train` 方法中，每个 `epoch` 训练时，在 `model.observe()` 内部进行参数更新，仅返回损失与准确率。最后，在 `_validate` 方法中，重新加载对应 `task_id` 对应的模型参数，并进行 `Shared` 的替换，再用该模型对该任务进行推理 (`inference`)。

3.4 Model

由于在 ACL 中 `backbone` 中的 `Private` 模块和 `head` 需要根据不同任务进行更换，暂时舍弃 `backbone` 等相关操作。可以考虑将 `AlexNet` 等作为 `backbone`，然后在 `Model` 中组合并手动记录 `task_id` 以设置模块的更换。另外因为 `Shared` 和 `Private` 的 `AlexNet` 结构本身会有差别。

在 `ACL.py` 中，定义模型 ACL 需要如下实现：

```
class Model(nn.Module)
```

- 记录 `self.task_id`
- `Shared`
- `Private`
- `head`
- `forward` : 根据 `task_id` 激活相关模块。

```
class `ACL`(Finetune)
```

- `__init__` : 初始化函数，设置算法所需的初始化参数。
- `observe` : 在训练阶段调用，输入一个批次的训练样本并返回预测结果、准确率和前向损失。面对一个 batch 的数据，需要进行对抗训练，包括训练 `Shared` 和 `Discriminator`。
- `inference` : 在推理阶段调用，输入一个批次的测试样本并返回分类结果和准确率。根据该批次数据的 `task_id` 选择对应的加载对应的模型，并将其中的 `Shared` 模块替换为现在的 `Shared` 模块。
- `before_task` : 在每个任务开始训练之前调用，用于调整模型结构、训练参数等，需要用户自定义。在这里设置 `model.task_id` 以及相关参数，确保在 `observe` 中激活正确模块。
- `after_task` : 在每个任务训练结束之后调用，用于调整模型结构、缓冲区。保存当前任务的模型参数，之后进行 `_validate` 会用到。
- `get_parameters` : 在每个任务开始训练之前调用，返回当前任务的训练参数。
- `after_epoch` : 在每一个 `epoch` 之后要进行 `eval_()` 验证，并根据结果进行学习率的调整。

四、复现结果

4.1 实验环境

- Windows 11
- Python 3.9
- Pytorch 1.12.0
- CUDA 12.3

4.2 实验结果

数据集：20个任务中分5类逐步学习CIFAR100数据集。

原文的实验结果为

Method	ACC%	BWT%	Arch (MB)	Replay Buffer (MB)
ACL (Ours)	78.08 (1.25)	0.00 (0.01)	25.1	-

我们的实验结果的准确率矩阵如下

	task1	task2	task3	task4	task5	task6	task7	task8	task9	task10	task11	task12	task13	task14	task15	task16	task17	task18	task19	task20
task1	72.57																			
task2	72.43	72.63																		
task3	72.43	72.81	79.06																	
task4	72.57	72.81	79.06	84.24																
task5	72.84	72.81	79.15	84.6	81.84															
task6	72.66	72.85	79.46	84.42	82.11	86.36														
task7	72.84	72.85	79.24	84.47	82.02	86.18	82.99													
task8	72.84	72.85	79.37	84.51	82.06	86.45	82.86	69.22												
task9	72.34	72.72	79.51	84.6	82.2	86.49	83.04	69.4	81.04											
task10	72.52	72.99	79.37	84.83	82.02	86.4	83.08	69.31	80.95	73.15										
task11	72.52	72.67	79.1	84.78	82.02	86.4	83.35	69.44	81.04	73.15	86.99									
task12	72.7	72.85	79.15	84.78	82.02	86.4	82.95	69.35	81.08	73.24	87.12	75.65								
task13	72.48	72.9	79.37	84.56	82.06	86.27	82.81	69.58	81.22	73.29	87.08	75.83	73.86							
task14	72.66	72.76	79.1	84.65	82.11	86.49	82.9	69.26	80.9	73.24	86.94	75.74	74.44	74.02						
task15	72.84	72.81	79.24	84.69	81.93	86.49	83.13	69.31	81.08	73.38	86.9	75.83	74.17	74.11	71.09					
task16	72.66	72.81	79.28	84.51	81.97	86.31	83.08	69.44	81.08	73.33	86.99	75.83	74.26	73.93	70.73	79.63				
task17	72.52	72.81	79.1	84.6	82.02	86.4	82.81	69.13	81.13	73.42	86.85	75.78	74.17	73.98	71.14	79.49	69.8			
task18	72.66	72.76	79.1	84.69	82.02	86.45	83.04	69.53	81.04	73.15	86.63	75.74	73.99	74.2	70.87	79.4	69.94	80.65		
task19	72.48	72.58	79.06	84.56	81.79	86.31	83.08	69.53	80.9	73.33	86.9	75.6	74.22	73.89	70.91	79.72	70.21	80.73	85.62	
task20	72.88	72.67	79.19	84.69	81.88	86.49	83.13	69.44	81.08	73.11	87.03	75.78	73.95	73.98	71.05	79.54	70.21	80.65	85.44	72.42

评价指标的数值为

$ACC = 77.73\%$ $BWT = 0.31\%$

可以看出，我们的复现已经达到原文的精度甚至略有超过。

4.2 日志

实验过程的日志可见 [LibContinual-ACL/log.txt at yk · KindofCrazy/LibContinual-ACL · GitHub](#)

五、遇到的问题与解决

5.1 数据加载的设置

在 `LibContinual` 框架的基础上，我们修改了每一个任务中的 `label` 使每一个任务的 `label` 均为 `[0,1,2,3,4]`，便于模型的验证。另外，在进行数据的处理时，添加属性 `tt` 与 `td` 分别表示 `task module labels` 和 `disctimator labels`。

5.2 backbone 与 buffer 的设置

原代码分别提供了 `AlexNet` 和 `ResNet` 作为 backbone 的 `Model`，但论文中只报告了将 `AlexNet` 作为 backbone 的实验结果，所以复现时只采用了将 `AlexNet` 作为 backbone 的 `Model`。

论文中对于 `CIFAR100` 数据集的训练细节的描述中说明了用了 `buffer` 操作，但在代码中并没有设置相关参数。所以我们在复现时并没有使用 `buffer`，最终对实验结果的精度没有影响。

5.3 训练

1. 初始化阶段，因为 `ACL` 与较为常用的训练流程有所区别，所以对于 `optimizer` 与 `lr_scheduler` 的设置需要手动完成。
2. 对于一个 `batch` 的数据，我们需要先训练 `Shared` 模块，再训练 `Discriminator` 模块，所以参数的更新需要在 `observe` 内部更新。
3. 因为在训练过程中需要保存模型，所以我们选择将模型保存在 `checkpoints` 文件夹下。

六 思考

实际上，持续学习中学习出不同任务中的共同特征，以及进一步识别出各自特征是非常直接的思路。本文非常具有创造力的想法是将对抗学习引入到持续学习来识别出不同任务中的共同特征，并利用潜在空间分解进一步识别出各自特征。这启示我们注重想法的交叉复合，尽可能抽象出任务的本质并与已存在其他的解决思路做对应，也许就会产生好的思路想法。

对于这次复现，我认为在以下方面存在改进的可能。

6.1 模型的设置

在我们对之前的任务的检验是将原来的模型加载进来然后替换其中 `Shared` 模块的情况下，也许没有必要保存原来的模型，只需要定义一个模型有一个 `Shared` 模块，一个包含了 `task_num` 个 `Private` 模块的模块以及 `task_num` 个 `head` 的模块。每次训练和验证时启用共同的 `Shared` 模块以及相应任务的 `Private` 模块和 `head` 即可，达到的效果应该是等价的，这样能大幅度减小存储要求。

6.2 Shared 模块的作用

在原作者代码的 `Issues` 中，一位开发者提出了以下问题。

```
I deactivated the gradients in the Shared module so that the weights do not change during learning. The accuracy on the tasks is still as good as with the activated gradient calculation of the Shared module. It seems to me that this module does not contribute to the classification and that the tasks are learned only on the Private modules.
```

这也引起我们思考，在学习的过程中，每一个模块起到了什么样的作用？这需要进一步的探索。